实 验 报 告

实验名称：实验四 LSTM 的实现

课程名称：认知科学与类脑识别

实验地点：K2

实验日期：2024.10.10

班 级：2022 级人工智能班

姓 名：薛润泽

学 号：202200130077

一 实验目的：

加深对 LSTM 模型的理解，能够使用 LSTM 模型解决简单问题

二 实验环境：

Windows

三 实验内容：

根据 LSTM 模型的相关知识，使用 Python 语言实现一个简单 LSTM 模 型。

实验要求：

（1） 随机产生 0-127 之间的两个八位的二进制整数，作为一组输 入数据，将这两个数的和作为一个标签，这三个数据组成一组训练数据，训练数据的组数应尽可能多。

（2） 创建 LSTM 网络。

（3） 实现两个八位的二进制整数的加法运算，网络能够输出正确 的加法运算结果。

四 实验步骤：

1. 实验程序：

（1）完成 LSTM 模型的初始化，因为设计到遗忘门、输入门、输出门，所以使用 Wfh,Wfx,bf 表示遗忘门超参数，Wih，Wix,bi 表示输入门超参数，Who，Wox，bo 表示输出门超参数，Wch，Wcx，Wbc 表示更新细胞记忆单元的超参数。

```
import os
import numpy as np
import random

class LstmLayer(object):
    def __init__(self, input_width, state_width, learning_rate):
        self.input_width = input_width
        self.state_width = state_width
        self.learning_rate = learning_rate
        self.gate_activator = SigmoidActivator()
        self.output_activator = TanhActivator()
        self.times = 0
        self.reset_state()
        self.init_weights()

    def init_weights(self):
        self.Wfh, self.Wfx, self.bf = self.init_weight_mat()
        self.Wih, self.Wix, self.bi = self.init_weight_mat()
        self.Woh, self.Wox, self.bo = self.init_weight_mat()
        self.Wch, self.Wcx, self.bc = self.init_weight_mat()
```

```
    def reset_state(self):
        self.times = 0
        self.c_list = [np.zeros((self.state_width, 1))]
        self.h_list = [np.zeros((self.state_width, 1))]
        self.f_list = [np.zeros((self.state_width, 1))]
        self.i_list = [np.zeros((self.state_width, 1))]
        self.o_list = [np.zeros((self.state_width, 1))]
        self.ct_list = [np.zeros((self.state_width, 1))]

    def init_weight_mat(self):
        Wh = np.random.uniform(-0.1, 0.1, (self.state_width, self.state_width))
        Wx = np.random.uniform(-0.1, 0.1, (self.state_width, self.input_width))
        b = np.zeros((self.state_width, 1))
        return Wh, Wx, b
```

（2）定义前向传播函数，输入一个自变量后，通过前向传播得到相应的细胞状态、隐藏层状态和输出状态

```
    def forward(self, x):
        self.times += 1
        fg = self.calc_gate(x, self.Wfx, self.Wfh, self.bf, self.gate_activator)
        self.f_list.append(fg)
        ig = self.calc_gate(x, self.Wix, self.Wih, self.bi, self.gate_activator)
        self.i_list.append(ig)
        og = self.calc_gate(x, self.Wox, self.Woh, self.bo, self.gate_activator)
        self.o_list.append(og)
        ct = self.calc_gate(x, self.Wcx, self.Wch, self.bc, self.output_activator)
        self.ct_list.append(ct)

        c = fg * self.c_list[self.times - 1] + ig * ct
        self.c_list.append(c)
        h = og * self.output_activator.forward(c)
        self.h_list.append(h)
```

（3）定义通用的门状态计算函数和更新函数

```
    def calc_gate(self, x, Wx, Wh, b, activator):
        h = self.h_list[self.times - 1]
        net = np.dot(Wh, h) + np.dot(Wx, x) + b
        gate = activator.forward(net)
        return gate
```

```
def update(self):
    self.Wfh -= self.learning_rate * self.Wfh_grad
    self.Wfx -= self.learning_rate * self.Wfx_grad
    self.bf -= self.learning_rate * self.bf_grad
    self.Wih -= self.learning_rate * self.Wih_grad
    self.Wix -= self.learning_rate * self.Wix_grad
    self.bi -= self.learning_rate * self.bi_grad
    self.Woh -= self.learning_rate * self.Woh_grad
    self.Wox -= self.learning_rate * self.Wox_grad
    self.bo -= self.learning_rate * self.bo_grad
    self.Wch -= self.learning_rate * self.Wch_grad
    self.Wcx -= self.learning_rate * self.Wcx_grad
    self.bc -= self.learning_rate * self.bc_grad
```

（4）定义初始化梯度函数

```python
def init_gradients(self):
    self.Wfh_grad = np.zeros_like(self.Wfh)
    self.Wfx_grad = np.zeros_like(self.Wfx)
    self.bf_grad = np.zeros_like(self.bf)
    self.Wih_grad = np.zeros_like(self.Wih)
    self.Wix_grad = np.zeros_like(self.Wix)
    self.bi_grad = np.zeros_like(self.bi)
    self.Woh_grad = np.zeros_like(self.Woh)
    self.Wox_grad = np.zeros_like(self.Wox)
    self.bo_grad = np.zeros_like(self.bo)
    self.Wch_grad = np.zeros_like(self.Wch)
    self.Wcx_grad = np.zeros_like(self.Wcx)
    self.bc_grad = np.zeros_like(self.bc)

def init_deltas(self):
    self.delta_f_list = self.init_delta_list()
    self.delta_i_list = self.init_delta_list()
    self.delta_o_list = self.init_delta_list()
    self.delta_ct_list = self.init_delta_list()

def init_delta_list(self):
    delta_list = []
    for i in range(self.times + 1):
        delta_list.append(np.zeros((self.state_width, 1)))
    return delta_list

def calc_delta(self, delta_h, activator):
    self.delta_h_list = self.init_delta_list()
    self.delta_h_list[-1] = delta_h
    for k in range(self.times, 0, -1):
        self.calc_delta_k(k)
```

```python
  def calc_delta_k(self, k):
      ig = self.i_list[k]
      og = self.o_list[k]
      fg = self.f_list[k]
      ct = self.ct_list[k]
      c = self.c_list[k]
      c_prev = self.c_list[k - 1]
      tanh_c = self.output_activator.forward(c)
      delta_k = self.delta_h_list[k]

      delta_o = delta_k * tanh_c * self.gate_activator.backward(og)
      delta_f = delta_k * self.delta_ct_list[k] * c_prev * self.gate_activator.backward(fg)
      delta_i = delta_k * self.delta_ct_list[k] * ct * self.gate_activator.backward(ig)
      delta_ct = delta_k * ig * self.output_activator.backward(tanh_c)

      delta_h_prev = (
          np.dot(self.Woh.T, delta_o) +
          np.dot(self.Wih.T, delta_i) +
          np.dot(self.Wfh.T, delta_f) +
          np.dot(self.Wch.T, delta_ct)
      )
      self.delta_h_list[k - 1] = delta_h_prev
```

（5）反向传播计算梯度，手动计算每次方向传播时的梯度，并进行梯度计算与传播

```
    self.delta_h_list[k - 1] = delta_h_prev

def calc_gradient(self, x):
    for t in range(self.times, 0, -1):
        Wfh_grad_t, bf_grad_t, Wih_grad_t, bi_grad_t, Woh_grad_t, bo_grad_t, Wch_grad_t, bc_grad_t = self.calc_gradient_t(t)
        self.Wfh_grad += Wfh_grad_t
        self.bf_grad += bf_grad_t
        self.Wih_grad += Wih_grad_t
        self.bi_grad += bi_grad_t
        self.Woh_grad += Woh_grad_t
        self.bo_grad += bo_grad_t
        self.Wch_grad += Wch_grad_t
        self.bc_grad += bc_grad_t

def init_weight_gradient_mat(self):
    Wh_grad = np.zeros((self.state_width, self.state_width))
    Wx_grad = np.zeros((self.state_width, self.input_width))
    b_grad = np.zeros((self.state_width, 1))
    return Wh_grad, Wx_grad, b_grad

def calc_gradient_t(self, t):
    h_prev = self.h_list[t - 1].transpose()
    Wfh_grad = np.dot(self.delta_f_list[t], h_prev)
    bf_grad = self.delta_f_list[t]
    Wih_grad = np.dot(self.delta_i_list[t], h_prev)
    bi_grad = self.delta_i_list[t]
    Woh_grad = np.dot(self.delta_o_list[t], h_prev)
    bo_grad = self.delta_o_list[t]
    Wch_grad = np.dot(self.delta_ct_list[t], h_prev)
    bc_grad = self.delta_ct_list[t]
    return Wfh_grad, bf_grad, Wih_grad, bi_grad, Woh_grad, bo_grad, Wch_grad, bc_grad
```

（6）定义批量生成 8 位 2 进制数的函数

```
def generate_binary_data(num_samples=1000):
    data = []
    for _ in range(num_samples):
        num1 = random.randint(0, 255)
        num2 = random.randint(0, 255)
        binary_num1 = f'{num1:08b}'
        binary_num2 = f'{num2:08b}'
        binary_sum = f'{(num1 + num2):08b}'[-8:]
        binary_num1_array = np.array([int(bit) for bit in binary_num1], dtype=np.float64).reshape(-1, 1)
        binary_num2_array = np.array([int(bit) for bit in binary_num2], dtype=np.float64).reshape(-1, 1)
        binary_sum_array = np.array([int(bit) for bit in binary_sum], dtype=np.float64).reshape(-1, 1)
        data.append((binary_num1_array, binary_num2_array, binary_sum_array))
    return data

def data_set():
    data = generate_binary_data(10000)
    x1 = np.array([item[0] for item in data])
    x2 = np.array([item[1] for item in data])
    d = np.array([item[2] for item in data])
    return [x1, x2], d
```

（7）定义 SIGMOD 函数和 Tanh 函数，用来做阶跃函数

```python
class SigmoidActivator(object):
    def forward(self, weighted_input):
        return 1.0 / (1.0 + np.exp(-weighted_input))

    def backward(self, output):
        return output * (1 - output)

class TanhActivator(object):
    def forward(self, weighted_input):
        return 2.0 / (1.0 + np.exp(-2 * weighted_input)) - 1.0

    def backward(self, output):
        return 1 - output * output
```

✓ 0.0s

（8）定义训练函数，暂定训练 500 轮次

```python
def train_lstm():
    lstm = LstmLayer(8, 8, 0.3)
    x, d = data_set()
    for epoch in range(500):
        for i in range(len(x[0])):
            lstm.reset_state()
            lstm.forward(x[0][i].reshape(-1, 1))
            lstm.forward(x[1][i].reshape(-1, 1))
            delta_h = lstm.h_list[-1] - d[i]
            lstm.backward(x[1][i].reshape(-1, 1), delta_h, TanhActivator())
            lstm.update()
    return lstm
def binary_to_array(binary_str):
    return np.array([int(bit) for bit in binary_str]).reshape(-1, 1)

def array_to_binary(array):
    binary_str = ''.join([str(int(bit > 0.5)) for bit in array])
    return binary_str
```

（9）定义测试函数，并进行训练和测试

```
def test_lstm(lstm, binary_num1, binary_num2):
    input1 = binary_to_array(binary_num1)
    input2 = binary_to_array(binary_num2)

    lstm.reset_state()
    lstm.forward(input1)
    lstm.forward(input2)

    output = lstm.h_list[-1]

    print(f"Input 1: {binary_num1}")
    print(f"Input 2: {binary_num2}")
    ansl =[]
    anslst = lstm.h_list[-1]
    for ansit in range(8):
        if anslst[ansit][0] > 0:
            ansl.append(1)
        else:
            ansl.append(0)
    print("PlusSum = ",end ='')
    for ansit2 in range(8):
        print(ansl[ansit2],end='')
    print()
    print(lstm.h_list[-1])
```

[71] ✓ 0.0s

```
lstm = train_lstm()
```

[49] ✓ 13m 1.9s

```
binary_num1 = "11000010"  # 194 in decimal
binary_num2 = "00100011"  # 35 in decimal

test_lstm(lstm, binary_num1, binary_num2)
```

[72] ✓ 0.0s

2. 程序实验结果：

程序输出结果为 test_lstm 的输出状态，即预测值。

```
            ansl.append(1)
        else:
            ansl.append(0)
    print("PlusSum = ",end ='')
    for ansit2 in range(8):
        print(ansl[ansit2],end='')
    print()
    print(lstm.h_list[-1])
```

[71] ✓ 0.0s

```
lstm = train_lstm()
```

[49] ✓ 13m 1.9s

```
binary_num1 = "11000010"  # 194 in decimal
binary_num2 = "00100011"  # 35 in decimal

test_lstm(lstm, binary_num1, binary_num2)
```

[72] ✓ 0.0s

```
Input 1: 11000010
Input 2: 00100011
PlusSum = 11100101
[[ 0.00861203]
 [ 0.00212479]
 [ 0.0186869 ]
 [-0.02339903]
 [-0.0096356 ]
 [ 0.07477494]
 [-0.01259191]
 [ 0.03253262]]
```

输出结果符合预期，加法器构造成功

3. 分析和改进：

（1）经过多次尝试，发现最后输出时不是单纯以 0 为界限的。比如有时候将>-0.2 的值转化为 1，<-0.2 的值转化为 0 才能满足加法器的输出要求。分析原因是因为没有定义好训练的输出域，没有搞清楚实

际输出的 LSTM 中 h 的具体意义。但是总体上满足一个线性可分的情况。

（2）有些样例会出现计算错误，对于同一个样本来说每一位预测正确率高。

五 实验小结：

模型构建中，最难处理的是反向传播求解梯度部分。

LSTM 是 RNN 神经网络的变形，比之 RNN 神经网络，LSTM 能更好的记忆远处的信息，因为它能“遗忘”不重要的信息。总体来说对于序列性长记忆场景 LSTM 效果更好。